JavaScript Graphics

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Présentation de Chart.js **3**

A. Présentation de Chart.js 3

B. Exercice : Quiz 10

III. Présentation de Plotly.js **11**

A. Présentation de Plotly.js 11

B. Exercice : Quiz 15

IV. Essentiel **16**

V. Auto-évaluation **16**

A. Exercice 16

B. Test 19

Solutions des exercices **20**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : ordinateur/tablette

Contexte

En tant que développeur web, il est important de maîtriser au moins un langage front end. Javascript est l'un d'eux. Votre expertise passe donc par la maîtrise d'un ensemble de composants, de librairies comme React ou comme celles qui vont être présentées lors des chapitres suivants.

Rappelons qu'une librairie est un ensemble d'outils qui permet aux développeurs web d'optimiser le temps de développement d'une application. Rappelons aussi que pour maîtriser une librairie, il peut être intéressant pour vous, de vous rendre sur la documentation de celle-ci : en ce qui concerne Plotly et Chart.js, vous trouverez les liens de ces documentations dans la partie 4 de votre cour.

L'objectif de ce cours est donc de comprendre concrètement comment utiliser les différentes librairies permettant de réaliser des graphiques dans des applications web. Nous évoquerons les modalités d'installation ainsi que leurs spécificités, ce qui les rapprochent et ce qui les séparent.

Dans le cadre de l'installation, il sera fait mention d'outils tels que NPM qui nécessitent d'installer Node.js. Vous trouverez le lien d'installation dans la dernière rubrique de ce cours.

Enfin, au titre des prérequis pour une bonne compréhension de ce cours, dans la mesure où il sera fait référence à des éléments tels que NPM, il est utile de disposer de bonnes connaissances sur Javascript.

[cf. 1_ Introduction à ChartJs _ Plotly Js.mp3]

II. Présentation de Chart.js

A. Présentation de Chart.js

Dans l'environnement Javascript, Chart.js et Plotly.js sont des librairies très populaires dont l'objet est de permettre la mise en place de graphique. Le rendu de ces graphiques peut s'effectuer en front ou en back.

Régulièrement, vous serez amené à faire un choix dans l'utilisation des librairies que vous allez utiliser. Ces choix vous amèneront à vous interroger sur :

- Le spectre de la librairie
- Le poids de la librairie et la maintenabilité de votre code

Le choix d'une librairie se fait en fonction de critères spécifiques, notamment :

- Sa popularité
- La date de sa dernière mise à jour
- Son nombre de téléchargement
- La compatibilité de la libraire dans l'environnement

Au regard de ces critères précis et factuels se dégagent 2 grandes librairies : Chart.js et Plotly.js. La connaissance d'au moins une de ces librairies peut s'avérer nécessaire dans un certain nombre de situations. Ce cours va vous présenter Chart.js.

Pourquoi l'utiliser ?

Vous serez confronté dans votre carrière à l'obligation d'enrichir vos pages web avec des graphiques, des éléments dynamiques, etc. C'est donc le moment de voir le fonctionnement et les avantages d'une librairie.

En ce qui concerne Chart.js, elle vous permettra d'assurer le rendu de vos graphiques en évitant des heures de codes et donc cela vous permettra d'optimiser votre temps de production. Comme cela sera détaillé plus tard, il est temps de faire un point sur votre environnement de travail.

Son environnement

Chart.js est une librairie Javascript qui est compatible avec l'ensemble des frameworks modernes comme React, Vue, Svelte, etc.

Mais aussi dans l'environnement Chart.js nous retrouvons des éléments HTML avec la balise **<canvas> </canvas>**. Nous reviendrons sur la balise Canvas ultérieurement.

C'est donc un environnement essentiellement front end que vous aurez à mettre en place.

Avant de procéder à l'installation de la librairie il est important que vous ayez l'habitude de naviguer dans la documentation pour approfondir et continuer l'apprentissage de cette librairie.

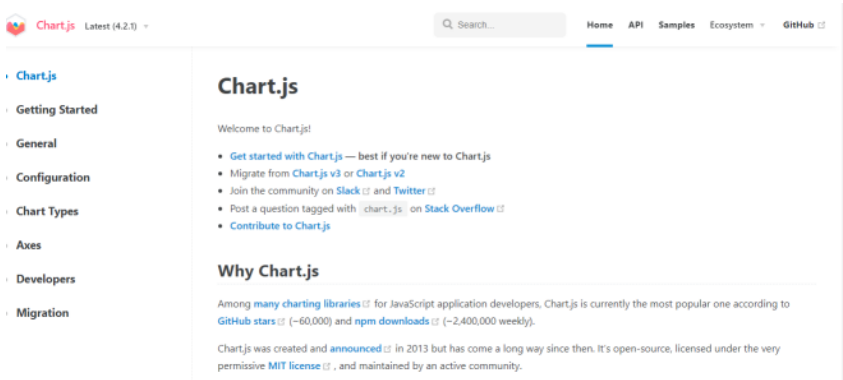
Le document est disponible sur le lien suivant : chartjs.org[1]

Tout d'abord à gauche de votre écran, vous vous apercevez qu'il existe une colonne dans laquelle figure un certain nombre d'onglets mais pour cette introduction nous nous bornerons aux onglets suivants :

- Getting Started
- Configuration
- Charts Types

L'onglet Getting Started est celui qui vous permettra de commencer votre projet et d'aller chercher les cdn et autres éléments nécessaires à l'installation. L'onglet Configuration sera plus important car il contient toutes les options dont vous aurez besoin, affichage du nom du graphique, etc. Enfin le dernier onglet est l'onglet qui vous permettra de retrouver l'ensemble des types de graphiques qui sont possibles.

Une attention toute particulière doit être portée à la version que vous vous apprêtez à télécharger, c'est pour cela que si vous observez la documentation vous pourrez constater qu'il existe un onglet déroulant reprenant la dernière version mais aussi les versions précédentes.



Page d'accueil du site Chart.js

1 https://www.chartjs.org/docs/latest/

Tous droits réservés à STUDI - Reproduction interdite

Source : chartjs.org[1]

Méthode

Il est possible d'installer Chart.js de différentes manière, notamment en utilisant webpack avec l'aide de cet import : import Chart from 'chart.js/auto';

Tout d'abord commençons par ce qui fait le quotidien des développeurs Javascript NPM. Comme vous l'avez appris, NPM est le gestionnaire de paquets pour Javascript.

Pour commencer l'installation, il est nécessaire de créer notre premier fichier HTML, pour cela vous devez ouvrir votre IDE et créer un fichier HTML.

```
<!DOCTYPE html>
<html>
<head>
<title>Chart Js Project</title>
</head>

<body>
The content of the document......
</body>

</html>
```

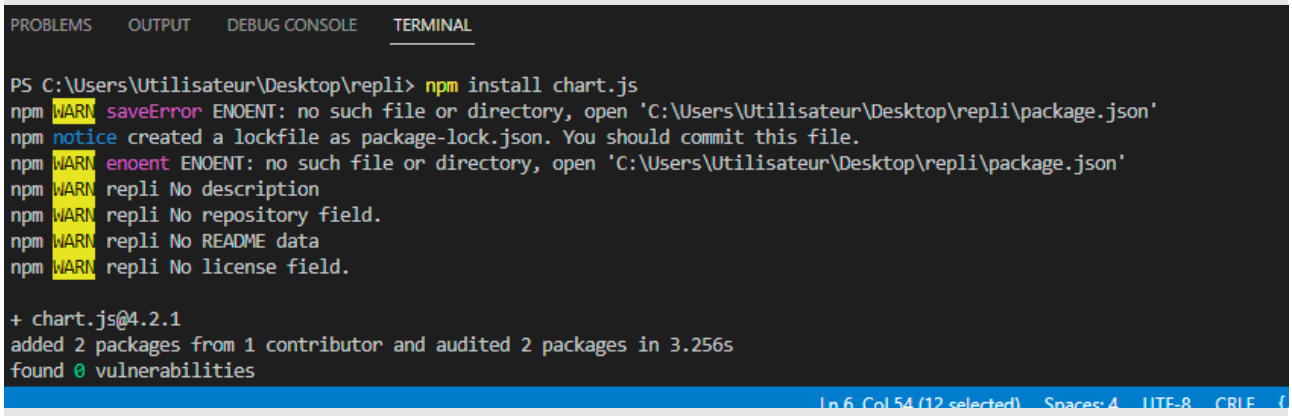
Dans la partie suivante, nous allons procéder à l'installation concrète de la librairie via NPM mais il sera nécessaire de vous assurer que Node.js est bien installé. Pour cela, vous devrez ouvrir votre invite de commande et taper node -v.. Normalement cela devrait vous afficher la version installée sur votre ordinateur.

Méthode **Installer Chart.js avec NPM**

Cette étape réalisée, il est maintenant impératif d'ouvrir votre terminal et de taper la commande suivante :

```
npm install chart.js
```

Vous devrez obtenir, si tout s'est bien passé, le visuel suivant :



Comme vous le voyez sur l'image ci-dessus, Chart.js est installé et 2 packages ont bien été installés. Vous constaterez d'ailleurs que dans l'arborescence de votre dossier, il existe maintenant un fichier node.

1 https://www.chartjs.org/

Tous droits réservés à STUDI - Reproduction interdite

Méthode

Cette installation a d'ailleurs généré un fichier Json dans l'arborescence de votre dossier. Si vous l'ouvrez, vous aurez ce qui suit :



Méthode **Installer Chart.js via un CDN**

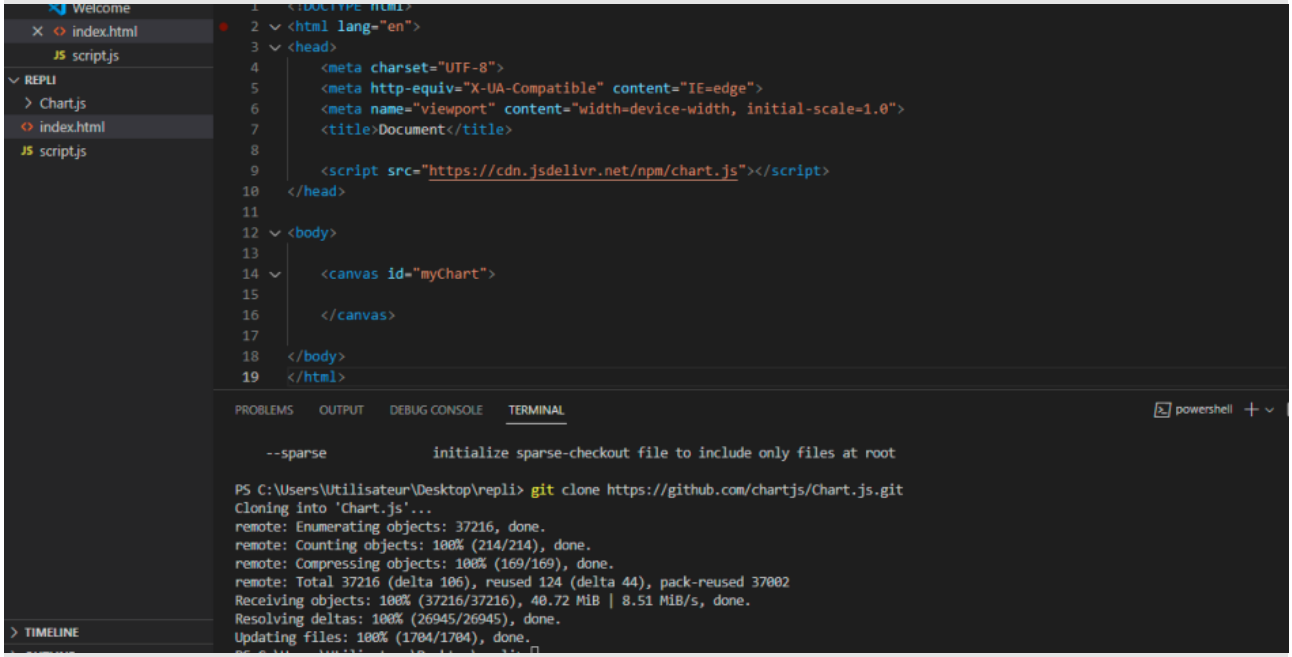
Cette installation ne présente aucune difficulté, car elle se limite à un simple copier-coller de liens figurant dans les bibliothèques suivantes. Ces liens doivent être copiés entre la balise **<head> </head>** et en fin de page avant la fermeture de la balise **<body>.** Pour disposer du lien, il est impératif d'aller sur la documentation et d'ouvrir l'onglet Getting Started.

Au titre des avantages de ce type d'installation, est soulignée la simplicité de l'installation qui se résume à un simple copier-coller du lien. L'inconvénient majeur est le fait que cela n'est qu'un lien, donc en cas de changement d'adresse du lien, votre application ne fonctionnera plus. Cela peut être particulièrement embêtant.

Méthode **Installer Chart.js via GitHub**

L'avantage d'une telle installation est la sécurité que cela engendre contrairement à un simple CDN, la librairie étant mise dans votre App, il n'y a pas de risque qu'elle soit déplacée et qu'elle impacte directement le rendu de vos graphiques. Pour procéder à l'installation via GitHub, il n'est pas nécessaire de disposer d'un compte GitHub, mais bien entendu, la maîtrise d'un instrument de versioning est un requis pour tout futur développeur. Pour télécharger le code, rendez-vous sur le lien contenu dans la documentation. Pour réaliser cette démarche, ouvrez un terminal comme le démontre l'image ci-après. L'installation via GitHub se fait par l'utilisation de la commande : **git clone**.

Tous droits réservés à STUDI - Reproduction interdite



Complément

Il est important de noter que lorsque Chart.js est téléchargé directement depuis le repository GitHub, celui-ci ne contient plus toutes les dépendances. Il est donc nécessaire de procéder à l'ajout d'un script autonome, en l'occurrence il s'agit de **pnpm**.

Pour cela il est nécessaire d'ouvrir un terminal et de taper la commande suivante :

```
iwr https://get.pnpm.io/install.ps1 -useb | iex
```

Cette commande vous permettra d'installer Chart.js et installera l'ensemble des dépendances nécessaires au bon fonctionnement de la librairie.

Méthode

Tout graphique réalisé grâce à Chart.js débutera par un certain nombre de nécessités et nous imposera un certain formalisme. Tout d'abord il nous faudra implanter une div sur notre fichier HTML qui contiendra un ID, que vous pourrez nommer comme vous le souhaitez. Pour la suite de ce cours et pour en faciliter la suite, l'ID sera identifié sous « *myChart* », mais il n'y aucune obligation de conserver la dénomination de l'ID.

```
<body>
<canvas id="myChart" width="400" height="100" aria-label="Hello ARIA World" role="img">
</canvas>
</body>
```

Comme vous le constatez, le départ du graphique se fait par la mise en place d'une balise **<canvas> </canvas>**. C'est une balise qui s'ouvre et qui se ferme, dédiée à l'environnement des graphiques. Prise seule, cette balise est une image et elle ne fournit aucune aide pour l'accessibilité, il est donc nécessaire de ne pas se servir seulement de cette balise pour répondre aux besoins d'accessibilité. L'élément canvas est un nouvel élément HTML qui est associé généralement à des scripts en Javascript. Il permet aux développeurs front de composer des graphiques et s'avère plus rapide au chargement.

Tous droits réservés à STUDI - Reproduction interdite

Complément **Le Javascript dans notre fichier HTML**

Pour simplifier la démarche et la compréhension, nous allons mettre en place Chart.js avec un lien CDN.

```
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

Le lien CDN est à mettre entre les balises **<head></head>**, il s'agit de la même démarche que celle qui consiste à utiliser le lien bootstrap. Le lien copié, il est maintenant nécessaire de mettre en place notre Javascript. Pour cela et pour simplifier le cours, nous utiliserons la balise script dans notre fichier HTML. Néanmoins il aurait été tout aussi possible d'injecter notre Javascript en créant un fichier dédié, appelé script.js. Sur notre fichier js figurera le code qui nous permettra de mettre en action.

```
<script>
let ctx = document.querySelector("#myChart");
          new Chart(ctx, {
            type: "pie",
            data: {
                labels: ['Paris', 'New-York', 'Madrid', 'Moscou', 'Berlin', 'Auckland'],
                datasets: [{
                  label: ' Nombre d'habitants',
                  data: [12, 19, 7, 5, 2, 3],
                  backgroundColor: ['red', 'blue', 'yellow', 'green', 'purple', 'orange']
              }]
            },
            options: {
                plugins: {
                   title: {
                    display: true,
                    text: 'Mon premier graphique avec Chart Js'
                 },
                },
                legend: {
                    position: 'bottom'
                }

}

});
</script>
```

Une lecture de notre code nous donnera les clés pour comprendre ce que nous venons de faire. Tout d'abord nous avons passé un **ID « *myChart* »** dans la balise canvas. Cet ID a été récupéré grâce à la méthode du document.querySelector. Nous avons ensuite utilisé new chart. Cette syntaxe est un impératif imposé par Chart.js.

Dans cette variable nous avons mis en place la balise **ctx** (context = getContext). Cette syntaxe est là aussi imposée par Chart.js. En effet, cette méthode nous est imposée par l'utilisation de la balise **<canvas>**.

Enfin, nous avons créé un objet dans lequel nous avons défini l'ensemble des données de notre graphique, son type, ses datas, etc.

Tous droits réservés à STUDI - Reproduction interdite

Configuration des options

Vous le remarquerez en lisant le code, il y a un ensemble d'options qui sont possibles, il existe d'autres types de graphiques possibles avec cette librairie.

Les types de graphiques possibles

Sans citer tous les graphiques, il est néanmoins possible d'en dégager trois ou quatre types que nous rencontrons régulièrement :

- Le radar
- Le graphique en ligne
- Le pie
- Le graphique en bar

Qu'est-ce que Data et Datasets ?

Data et datasets sont 2 éléments incontournables de Chart.js. Data est avant tout un objet qui contiendra un ensemble de données telles que décrit dans l'exemple ci-après. Cet exemple vous montre ce que sont les datas dans chart.js et ce qu'il contient.

```
type: 'line',
   data: {
     labels: ['Paris ', 'London', 'New York', 'Singapore', 'Shanghai', 'Sydney'],
```

Dans cet objet sera inclus l'ensemble des variables qui feront l'objet d'un graphique.

Pour ce qui est de Datasets, il s'agit de transcrire une donnée qui est un tableau qui contiendra des objets tels que vous le voyez ci-dessous.

```
datasets: [{
        label: 'Nombre de résident en million',
        data: [12, 19, 3, 5, 2, 3],
        borderWidth: 1
      }]
```

Datasets contiendra donc les variables suivantes :

- Les labels qui sont des chaînes de caractères,
- Les data qui sont des nombres,
- Et pour finir des éléments de css comme la taille et la couleur d'une ligne de point.

Les options possibles à paramétrer

Une option est un objet dans lequel nous plaçons d'autres objets comme le montre le code ci-dessous.

Une option particulièrement intéressante est **chartContainer** qui permettra d'assurer les aspects responsive de votre graphique.

```
<div class="chart-container">
    <canvas id="myChart"></canvas>
</div>
```

L'inconvénient du genre de structure proposé par Chart.js se situe au niveau de la maintenabilité du code, dès qu'un graphique s'avère plus complexe. En effet, chaque option se déclare dans un objet. De fait, si le graphique est complexe, le code peut vite devenir compliqué à lire. Comme le montre le code ci-après.

```
options: {
                title: {
                    display: true,
                    text: 'Mon premier graphique avec Chart Js'
                },
```

Tous droits réservés à STUDI - Reproduction interdite

```
            legend: {
                position: 'bottom'
            }
        }
```

Remarque

Les graphiques développés via Chart.js ne commencent pas au zéro sur les axes des X et Y, il peut donc être important de paramétrer le début de la numération en paramétrant le y comme débutant à 0.

B. Exercice : Quiz

[solution n°1 p.21]

Question 1

Pour assurer les aspects responsifs sur Chart.js, vous pouvez :

- ○ Utiliser la classe « *ChartContainer* »
- ○ Utiliser le système des options avec un display True ?
- ○ Les deux propositions précédentes sont nécessaires
- ○ Rien du tout, Chart.js est nativement responsif

Question 2

Quelle est la syntaxe pour la balise canvas ?

- ○ <canvas/>
- ○ <canvas></canvas>
- ○ canvas

Question 3

Quelle est la syntaxe pour configurer le type de graphique ?

- ○
```
const cfg = {
  type: 'bar',
  data: {
    datasets: [{
      data: [20, 10],
    }],
    labels: ['a', 'b']
  }
}
```


Tous droits réservés à STUDI - Reproduction interdite

- ```
  const cfg = {
    type: 'bar',
    data: [{
   datasets: [{
        data: [20, 10],
    }],
      labels: ['a', 'b']
    }
  }
  ```
- ```
  const cfg = {
    type: 'bar';
    data: {
      datasets: [{
        data: [20, 10]
     }];
      labels: ['a', 'b']

    }
  }
  ```

Question 4

Lors de l'installation de la librairie Chart.js via NPM, quelle commande devez-vous entrer ?

- Npm-install chart js
- Npm_install chart.j
- npm install chart.js
- npm install -i -chartjs

Question 5

Le clonage du repository de GitHub permet de faire fonctionner la librairie avec toutes ses dépendances.

- Vrai
- Faux

III. Présentation de Plotly.js

A. Présentation de Plotly.js

[cf. 4_Présentation de Plotly.mp3]

Autre librairie spécialisée dans la création de graphiques : Plotly.js.

Installer Plotly.js

Comme Chart.js, Plotly.js est une librairie qui viendra vous aider à enrichir vos pages de graphiques responsives et dynamiques. Cette librairie est aussi open source. L'une des grandes différences se situera dans les modalités d'installation.

De plus, elle a la caractéristique d'être disponible pour de nombreux langages de programmation.

Tous droits réservés à STUDI - Reproduction interdite

Il y a bien entendu la possibilité d'installer Plotly.js via NPM. Pour cela il est nécessaire d'ouvrir votre terminal et de taper la commande suivante :

```
npm i plotly.js
```

Elle s'installe aussi via le cdn, ainsi que via le repos de GitHub. Ces démarches ayant été vues en amont avec l'introduction de Chart.js, il est plus important de voir la dernière façon d'installer cette librairie, via webpack.

Conseil **Webpack pour l'installation de Plotly.js dans un projet HTML**

Il est possible d'installer Plotly.js via webpack.

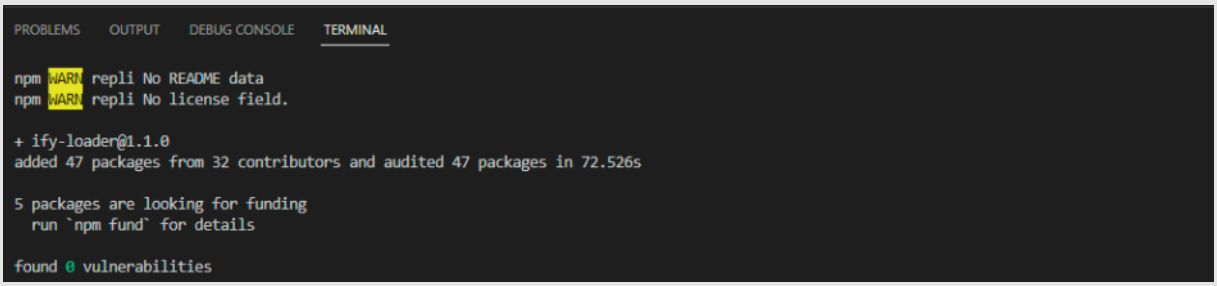
La procédure sera donc un peu plus complexe si vous souhaitez utiliser webpack pour installer cette librairie, néanmoins, une fois maîtrisée, elle est assez simple à mettre en place.

Méthode

L'installation via webpack nécessite un certain nombre de démarches, la première est l'installation de **ify-loader** en utilisant npm.

```
npm install --save ify-loader
```

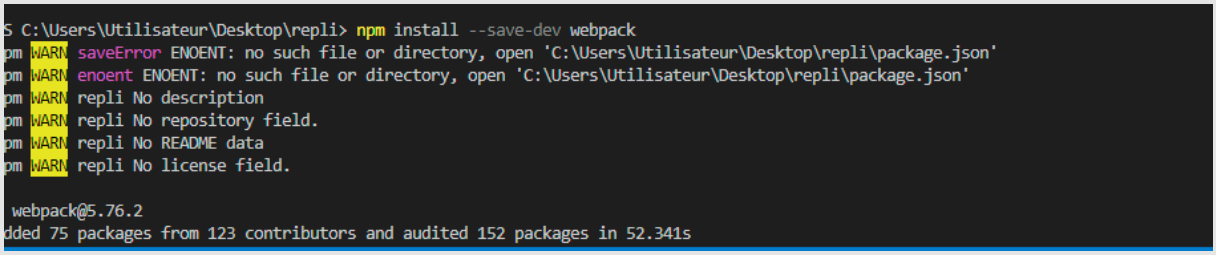
L'installation de cette dépendance devrait vous donner le visuel suivant :



Ify -loader téléchargé, il sera nécessaire d'installer webpack dans votre projet :

```
npm install --save-dev webpack
```

Si l'installation s'est bien déroulée, vous devriez avoir le visuel suivant :




```
npm install --save-dev webpack
```

Enfin, vous devrez configurer webpack en ajoutant un script à votre webpack.config.json. Il vous sera donc nécessaire d'ajouter le code ci-dessous.

```
...
      module: {
          rules: [
              {
                  test: /\.js$/,
                  loader: 'ify-loader'
              }
          ]
```


Tous droits réservés à STUDI - Reproduction interdite

```
    },
...
```

Les impératifs : une balise HTML pour le départ

Comme pour Chart.js, la première démarche est la création d'un fichier HTML. Entre les 2 balises **<body></body**>, il est nécessaire de mettre une balise ** **.

Méthode

```
<div id="tester" style="width:600px;height:250px;"></div>
```

Cette ** ** installée, il sera nécessaire de créer notre fichier Javascript sur lequel l'ensemble des données sera injecté.

```
<script>
  let graphDiv = document.getElementById('tester')

  let data = [{
    x: [1999, 2000, 2001, 2002],
    y: [10, 15, 13, 17],
    type: 'scatter'
  }];

  let layout = {
    title: 'Sales Growth',
    xaxis: {
      title: 'Year',
      showgrid: false,
      zeroline: false
    },
    yaxis: {
      title: 'Percent',
      showline: false
    }
  };
  Plotly.newPlot(graphDiv, data, layout);
  let dataRetrievedLater = graphDiv.data;
  let layoutRetrievedLater = graphDiv.layout;

</script>
```

Conseil

Plotly.newPlot(graphDiv, data, layout); cette classe est un impératif dans Plotly.js : elle s'introduit toujours dans une ** **. Elle contient l'ensemble des éléments du graphiques, comme les données, le titre, l'axe des x et des y, etc.

La variable layout est incluse dans un objet, l'objet layout servira à stocker des variables comme le titre, le départ à zéro du graphique, etc.

Tous droits réservés à STUDI - Reproduction interdite

Comment configurer les options du graphique

Il est possible de paramétrer l'ensemble du graphique pour obtenir un rendu correspondant à nos attentes. La librairie comptant plus de 40 types de graphiques, il est plus intéressant de s'attarder sur la méthode pour ajouter une ou plusieurs options. Pour cela, il sera nécessaire de déclarer une variable qui dans un objet contiendra les options voulues, telle que le nom, le responsive, etc. Observez le code ci-dessous, il a pour objectif de permettre un zoom et un scroll.

Méthode

```
<script>
    let graphDiv = document.getElementById('tester')

  let data = [{
    x: [1999, 2000, 2001, 2002],
    y: [10, 15, 13, 17],
    type: 'scatter'
  }];

  let layout = {
    title: 'Sales Growth',
    xaxis: {
      title: 'Year',
      showgrid: false,
      zeroline: false
    },
    yaxis: {
      title: 'Percent',
      showline: false
    }
  };
  Plotly.newPlot(graphDiv, data, layout, );
  let dataRetrievedLater = graphDiv.data;
  let layoutRetrievedLater = graphDiv.layout;

</script>
```

La mise en place des options se fait dans une variable « *Layout* », dans l'exemple au-dessus, il s'agit de l'option scroll et zoom mais la démarche serait la même avec des aspects responsifs.

Exemple

Voici un exemple de projet avec Plotly.js.

```
<!DOCTYPE html>
<html lang="fr">

<head>
<!-- cdn Plotly.js entre les balises head -->
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
</head>
<body>
<!-- Plots s'installe dans une div.-->
<div id="tester" style="width:90%;height:250px;"></div>
<script src="script.js"></script>
</body>
</html>
```


Tous droits réservés à STUDI - Reproduction interdite

```
<script>
let tester = document.getElementById('tester');
Plotly.newPlot( tester, [
  {
    x: [1, 2, 3, 4, 5],
    y: [1, 2, 4, 8, 16]
  }
],
  {
    margin: { t: 0 }
});
<script>
```

L'utilisation de ces morceaux de codes vous permettront de réaliser votre premier graphique en ligne avec Plotly.js.

B. Exercice : Quiz

[solution n°2 p.22]

Question 1

Parmi les propositions suivantes, laquelle est la syntaxe utilisée par Plotly.js lors de la mise en place d'un projet ?

- Plotly.newPlot
- Plotlynew.Plot
- PlotlynewPlot

Question 2

Dans le cadre d'un projet faisant appel à Plotly.js, quelle balise devez-vous utiliser lors de l'initiation d'un projet Plotly ?

- Une div
- Une div avec la balise canvas
- Une div avec un id

Question 3

Quelle est la syntaxe pour configurer le type de graphique ?

-
```
let layout = {
title: 'Scroll and Zoom',
showlegend: false };
```

-
```
let layout = [
 title: 'Scroll and Zoom',
 showlegend: false ];
```

-
```
let layout = [{
 title: 'Scroll and Zoom',
 showlegend: false }];
```

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Lors de l’installation de la librairie Plotly.js s via NPM, quelle commande devez-vous entrer ?

- Npm-install plotly js
- npm i plotly.js
- npm install plotly.js-dist
- npm install plotly-dist.js

Question 5

Le clonage du repository de GitHub permet de faire fonctionner la librairie avec toutes ses dépendances.

- Vrai
- Faux

IV. Essentiel

Il est à retenir un certain nombre d'éléments de ces deux librairies. Elles sont toutes deux en open source et donc complètement gratuites. Elles sont aussi toutes deux totalement customisables grâce au système des options.

Elles s’installent de la même façon (NPM, GitHub, CDN), à l'exception près que Plotly.js peut s’installer via webpack. Elles se mettent dans une balise de type div dans un fichier HTML.

De plus, toutes les deux nécessitent la mise en place d’un ID qui est récupéré par document.querySelector ou document.getElementById.

Ces librairies sont totalement customisables et offrent l’opportunité de mettre en place rapidement et très simplement des graphiques que cela soit la partie front end ou la partie back end. Elles prennent aussi en compte les aspects responsives nécessaires compte tenu de l’évolution des appareils connectés.

Ces librairies prennent en considération l’expérience utilisateur et notamment l’accessibilité pour les personnes souffrant d’une déficience visuelle. Chart.js ajoute un rôle <img> dans le canvas déclaratif du fichier HTML.

V. Auto-évaluation

A. Exercice

Vous êtes reçu par une agence web dans le cadre d’un entretien technique. Votre futur manager vous confie votre première mission. Il vous demande de réaliser deux graphiques avec deux libraires spécialisées : Chart.js et Plotly.js. Avec Chart.js, il vous demande de créer un graphique de type pie.

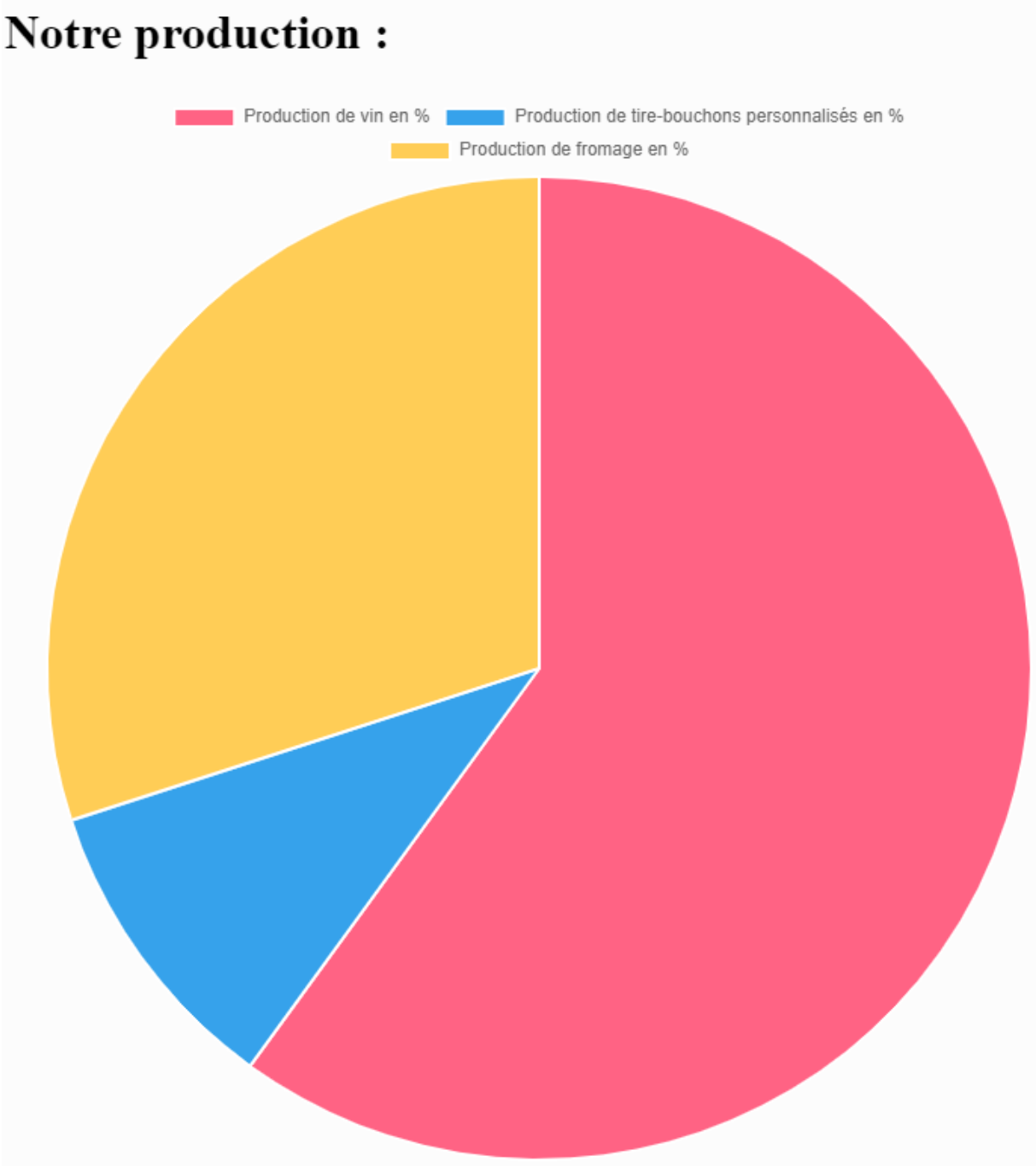
Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°3 p.23]

Dans un premier temps, votre patron vous demande de créer un graphique de type pie avec Chart.js pour une société proposant des dégustations de vins et de fromages de la région. Ceux-ci souhaitent faire apparaître leur pourcentage de production de vin, de tire-bouchons personnalisés et de fromages.

Voici ce que vous devriez obtenir :



Voici les données à renseigner :

- 60 % de vin
- 10 % de tire-bouchons
- 30 % de fromage

Tous droits réservés à STUDI - Reproduction interdite

Enfin, voici le code HTML :

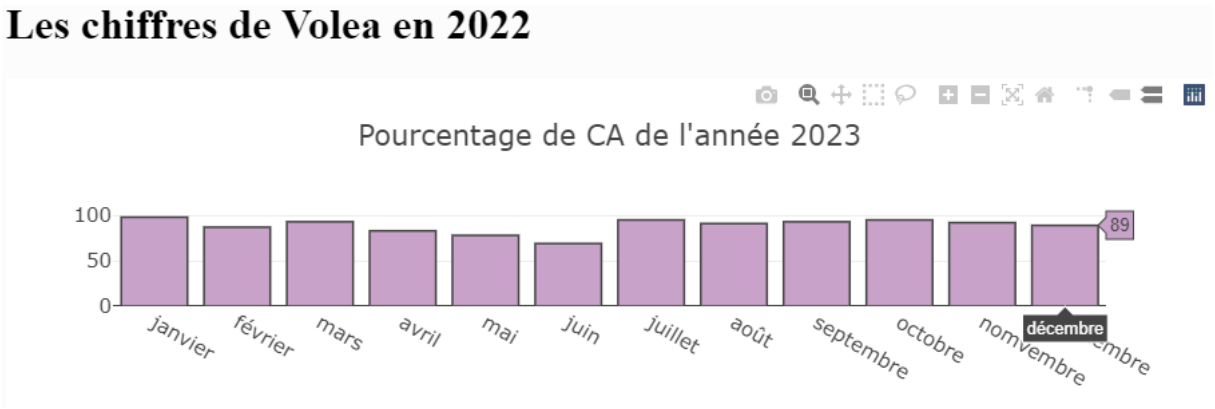
```
<!DOCTYPE html>
<html lang="fr">
<head>
 <!-- Mis en place du cdn   -->
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
<h1>Notre production :</h1>
<!-- Chart s'installe entre des éléments de div.-->
<div>
<canvas id="myChart"></canvas>
</div>
<script src="script.js"></script>
</body>
</html>
```

Question 2

[solution n°4 p.24]

Dans un second temps, votre patron vous demande de créer un graphique de type bar tout en prenant en compte l'aspect responsive dans le cadre du projet d'un client. Ce client souhaite indiquer sous la forme d'un diagramme le pourcentage de chiffre d'affaires atteint par l'entreprise Volea en fonction de son chiffre d'affaires prévisionnel.

Voici ce que vous devriez obtenir :



Voici les données à renseigner :

- Janvier : 98 %
- Février : 87 %
- Mars : 93 %
- Avril : 83 %
- Mai : 78 %
- Juin : 69 %
- Juillet : 95 %
- Août : 91 %
- Septembre : 93 %
- Octobre : 95 %
- Novembre : 92 %
- Décembre : 89 %

Tous droits réservés à STUDI - Reproduction interdite

Enfin, voici le code HTML :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <!-- Mis en place du cdn Plotly Js  -->
  <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
</head>
<h1>Les chiffres de Volea en 2022</h1>
<!-- Plots s'installe entre des éléments de div.
-->
<div id="tester" style="width:90%;height:250px;"></div>
<script src="script.js"></script>
</body>
</html>
```

B. Test

Exercice 1 : Quiz

[solution n°5 p.24]

Question 1

Il existe une différence majeure entre Plotly.js et Chart.js dans les méthodes d'installation.

- ○ Vrai
- ○ Faux

Question 2

Il n'est pas possible d'utiliser Plotly.js avec des frameworks modernes tels que Vue.js et React.

- ○ Vrai
- ○ Faux

Question 3

Dans le cas d'un projet Chart, la notion d'accessibilité peut poser problème, il est recommandé de doter la balise canvas…

- ○ D'un width
- ○ D'un height
- ○ D'un width, d'un height, d'un rôle avec une balise img

Question 4

Dans quel élément Javascript se déclare le type de graphique ?

- ○ Un tableau
- ○ Un objet
- ○ Un tableau contenant un objet

Question 5

C'est un inconvénient de déclarer un rôle <img> dans l'insertion de la balise Chart.

- ○ Vrai
- ○ Faux

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 10 **Solution n°1**

Question 1

Pour assurer les aspects responsifs sur Chart.js, vous pouvez :

- ◉ Utiliser la classe « *ChartContainer* »
- ○ Utiliser le système des options avec un display True ?
- ○ Les deux propositions précédentes sont nécessaires
- ○ Rien du tout, Chart.js est nativement responsif

Dans le cadre d'un projet Chart.js, il est nécessaire d'utiliser la classe « *ChartContainer* » qui permettra de rendre le graphique responsive.

Question 2

Quelle est la syntaxe pour la balise canvas ?

- ○ <canvas/>
- ◉ <canvas></canvas>
- ○ canvas

Canvas est une balise HTML fermante de ce fait, la seule solution valide est <canvas></canvas>.

Question 3

Quelle est la syntaxe pour configurer le type de graphique ?

- ◉
```
const cfg = {
  type: 'bar',
  data: {
    datasets: [{
      data: [20, 10],
    }],
    labels: ['a', 'b']
  }
}
```
- ○
```
const cfg = {
  type: 'bar',
  data: [{
 datasets: [{
      data: [20, 10],
  }],
    labels: ['a', 'b']
  }
}
```

Tous droits réservés à STUDI - Reproduction interdite

```
const cfg = {
  type: 'bar';
  data: {
    datasets: [{
      data: [20, 10]
   }];
    labels: ['a', 'b']

  }
}
```

Data est un objet, ce qui le diffère de datasets, qui est un tableau qui contient des objets.

Question 4

Lors de l'installation de la librairie Chart.js via NPM, quelle commande devez-vous entrer ?

- Npm-install chart js
- Npm_install chart.j
- npm install chart.js
- npm install -i -chartjs

La bonne réponse est npm install chart.js. Cette installation se fait en ligne de commande via le terminal de votre IDE. Cette commande est la seule permettant l'installation de Chart.js via NPM, elle est donc à connaître.

Question 5

Le clonage du repository de GitHub permet de faire fonctionner la librairie avec toutes ses dépendances.

- Vrai
- Faux

Le simple clonage du repository de Chart.js ne permettra pas le fonctionnement de la librairie avec toutes ses dépendances. Il faudra installer un script autonome usuellement pnpm.

Exercice p. 15 **Solution n°2**

Question 1

Parmi les propositions suivantes, laquelle est la syntaxe utilisée par Plotly.js lors de la mise en place d'un projet ?

- Plotly.newPlot
- Plotlynew.Plot
- PlotlynewPlot

C'est la syntaxe utilisée par Plotly.js pour créer un nouveau graphique dans une page HTML.

Question 2

Dans le cadre d'un projet faisant appel à Plotly.js, quelle balise devez-vous utiliser lors de l'initiation d'un projet Plotly ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Une div
- ○ Une div avec la balise canvas
- ◉ Une div avec un id

Canvas est une balise HTML, malgré cela, elle n'est pas utilisée par Plotly.js qui préfère installer une simple div.

Question 3

Quelle est la syntaxe pour configurer le type de graphique ?

- ◉
```
let layout = {
title: 'Scroll and Zoom',
showlegend: false };
```
- ○
```
let layout = [
 title: 'Scroll and Zoom',
 showlegend: false ];
```
- ○
```
let layout = [{
 title: 'Scroll and Zoom',
 showlegend: false }];
```

La variable layout s'inscrit dans un objet.

Question 4

Lors de l'installation de la librairie Plotly.js s via NPM, quelle commande devez-vous entrer ?

- ○ Npm-install plotly js
- ◉ npm i plotly.js
- ○ npm install plotly.js-dist
- ○ npm install plotly-dist.js

La bonne réponse est npm i plotly.js. Cette installation se fera en ligne de commande via votre IDE. Là aussi vous pouvez retrouver la commande sur le lien suivant : plotly.js[1]

Question 5

Le clonage du repository de GitHub permet de faire fonctionner la librairie avec toutes ses dépendances.

- ◉ Vrai
- ○ Faux

Plotly.js s'installe aussi via GitHub sans difficulté particulière. Cette installation se fait en ligne de commande via votre IDE. Il sera nécessaire de taper la commande suivante, git clone plotly/plotly.js[2].

Cette installation vous permettra de télécharger la librairie dans votre projet.

p. 17 **Solution n°3**

1 https://www.npmjs.com/package/plotly.js
2 https://github.com/plotly/plotly.js.git

Tous droits réservés à STUDI - Reproduction interdite

```
<script>
let pieChart = document.querySelector("#myChart");
          new Chart(pieChart, {
            type: "pie",
            data: {
                labels: ['Production de vin en %', 'Production de tire-bouchons personnalisés en %', 'Production de fromage en %'],
                datasets: [{
                  data: [60, 10, 30],
                  backgroundColor: [
                    'rgb(255, 99, 132)',
                    'rgb(54, 162, 235)',
                    'rgb(255, 205, 86)'
                  ],
                hoverOffset: 4
                }]
            }
          });
</script>
```

p. 18 **Solution n°4**

```
<script>
let tester = document.getElementById('tester')
let data = [{
  type: 'bar',
  x: ['janvier','février', 'mars', 'avril', 'mai','juin','juillet', 'août', 'septembre', 'octobre', 'nomvembre', 'décembre'],
  y: [98, 87, 93, 83, 78, 69, 95, 91, 93, 95, 92, 89],
  marker: {
      color: '#C8A2C8',
      line: {
          width: 1.5
      }
  }
}];
let layout = {
  title: 'Pourcentage de CA de l\'année 2022',
  font: {size: 15}
};
let config = {responsive: true}
Plotly.newPlot(tester, data, layout, config );
</script>
```

Exercice p. 19 **Solution n°5**

Question 1

Il existe une différence majeure entre Plotly.js et Chart.js dans les méthodes d'installation.

- ○ Vrai
- ◉ Faux

Il est possible d'installer Plotlys.js et Chart.js via webpack.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Il n'est pas possible d'utiliser Plotly.js avec des frameworks modernes tels que Vue.js et React.

- ○ Vrai
- ◉ Faux

Plotly et Chart sont compatibles avec les framework modernes. Pour Plotly, il faudra créer un composant.

Question 3

Dans le cas d'un projet Chart, la notion d'accessibilité peut poser problème, il est recommandé de doter la balise canvas…

- ○ D'un width
- ○ D'un height
- ◉ D'un width, d'un height, d'un rôle avec une balise img

Dans le cadre d'un projet sous Chart.js, l'accessibilité peut poser des soucis, donc pour un meilleur référencement, une bonne pratique est de doter la balise canvas d'un width, d'un height, d'un rôle avec une balise img.

Question 4

Dans quel élément Javascript se déclare le type de graphique ?

- ○ Un tableau
- ◉ Un objet
- ○ Un tableau contenant un objet

Il se déclare dans un objet qui pourra contenir un certain nombres de valeurs, comme un tableau qui lui-même contiendra les valeurs attribuées à x et y.

Question 5

C'est un inconvénient de déclarer un rôle <img> dans l'insertion de la balise Chart.

- ○ Vrai
- ◉ Faux

L'accessibilité pour les personnes non voyante ou ayant une déficience est un élément important pour le référencement d'un site. C'est pourquoi il est judicieux d'insérer un rôle dans le canvas.

Tous droits réservés à STUDI - Reproduction interdite